



Institut International d'Ingenierie de l'Eau et de l'Environnement

Master Intelligence Artificielle — Cloud Computing & Securite

Livrable 2

Projet EduScore

Evaluation du Risque de Decrochage Scolaire

Groupe : 1

Plateforme : AWS, region eu-west-1

Repository : https://github.com/ZANA-JB/Projet_securitecloud_used

Date : 14 juin 2026

URL publique de l'application

<https://d3oufpgazy9f3g.cloudfront.net>

Services : Frontend React | API FastAPI | PostgreSQL | Modele ML

Table des matières

1	Contexte et objectif	2
2	Architecture finale	2
2.1	Schema de l'architecture deployee	2
2.2	Comparaison avec l'architecture prevue (Livrable 1)	3
3	Developpement	3
3.1	Backend — FastAPI	3
3.1.1	Modele ML — Features et classification	3
3.1.2	Validation des entrees	4
3.2	Frontend — React	4
3.3	Base de donnees — PostgreSQL	5
4	Conteneurisation et deploiement	6
4.1	Docker et Docker Compose	6
4.2	Pipeline CI/CD — GitHub Actions	7
4.3	Deploiement sur AWS	8
5	Securite	9
5.1	Execution du plan de securite (Livrable 1)	9
5.2	Gestion des secrets	9
6	Couts	9
6.1	Couts reels vs estimates	10
7	Difficultes rencontrees et solutions	11
8	Conclusion	11
	Annexes	12

1. Contexte et objectif

EduScore est une plateforme d'aide à la décision destinée aux établissements scolaires, permettant d'identifier les élèves susceptibles de décrocher. Tel que défini dans le Livrable 1, le système prédit l'une des trois catégories — **En réussite**, **A surveiller** ou **A risque de décrochage** — à partir de données saisies par l'enseignant : nom de l'élève, niveau scolaire, filière, note moyenne, taux d'assiduité, heures d'étude par semaine, accès aux ressources et résultat au dernier examen.

Ce Livrable 2 documente le passage de la conception à l'exécution : l'application est déployée et accessible publiquement à l'adresse <https://d3oufpgazy9f3g.cloudfront.net>, les pipelines CI/CD sont actifs et les mesures de sécurité planifiées dans le Livrable 1 ont été mises en œuvre.

2. Architecture finale

2.1. Schema de l'architecture déployée

L'architecture déployée reprend la **variante budget** définie dans le Livrable 1, avec l'ajout de CloudFront comme CDN et point de terminaison HTTPS.

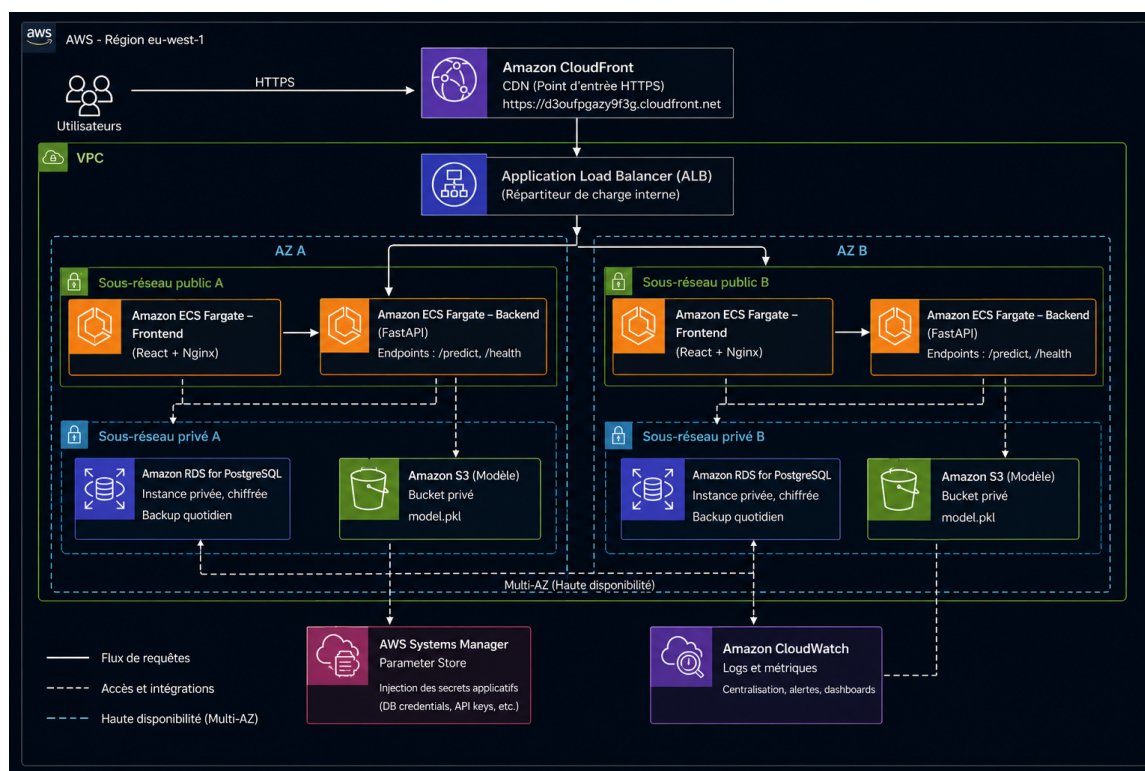


FIGURE 1 – Architecture AWS EduScore — déploiement final (Groupe 1)

Les composants principaux sont :

- **CloudFront** : CDN AWS servant de point d'entrée HTTPS (<https://d3oufpgazy9f3g.cloudfront.net>).
- **ALB (Application Load Balancer)** : répartiteur de charge interne, réparti sur deux zones de disponibilité (AZ A / AZ B).
- **ECS Fargate — Frontend** : conteneur React + Nginx.
- **ECS Fargate — Backend** : conteneur FastAPI, endpoints `/predict` et `/health`.
- **PostgreSQL RDS** : instance en sous-réseau privée, chiffrée, avec backup quotidien.

- **S3 (modele)** : bucket prive stockant `model.pkl`.
- **SSM Parameter Store** : injection des secrets applicatifs.
- **CloudWatch** : centralisation des logs et metriques.

2.2. Comparaison avec l'architecture prevue (Livrable 1)

TABLE 1 – Comparaison architecture prevue vs reelle

Composant	Prevu (L1)	Realise (L2)
Frontend	ECS Fargate React+Nginx	Oui — identique
Backend	ECS Fargate FastAPI	Oui — identique
Base de donnees	PostgreSQL RDS prive	Oui — identique
Modele ML	S3 prive chiffre	Oui — charge au demarrage
CloudFront	Non prevu (L1)	Oui — ajoute (CDN + HTTPS)
ALB / HTTPS	ALB multi-AZ + HTTPS	Oui — certificat ACM
SSM Secrets	AWS SSM Parameter Store	Oui — JWT, DB URL injectes
JWT Auth	Google OAuth + JWT	Oui — effectué
NAT Gateway	Optionnel (budget)	Non — non deploye, budget
CloudWatch	Logs + metriques	Oui — dashboards actifs

3. Developpement

3.1. Backend — FastAPI

Le backend est developpe en **Python 3.11** avec le framework **FastAPI**. Il expose les endpoints suivants :

- **GET /health** : retourne `{"status": "ok"}` pour les health checks de l'ALB Target Group.
- **POST /predict** : recoit un objet JSON, valide via **Pydantic v2**, interroge le modele ML et renvoie la categorie predite avec le score de confiance.
- **GET /history** : retourne l'historique des predictions depuis PostgreSQL.

3.1.1. Modele ML — Features et classification

Le modele est un **classifieur multi-classe** (Random Forest) entraine avec **scikit-learn**. Les features correspondent exactement aux champs du formulaire :

TABLE 2 – Features du modele de prediction EduScore

Feature	Type	Description / Valeurs
<code>nom_eleve</code>	string	Identifiant (stocke pour l'historique, non utilise dans le modele)
<code>niveau_scolaire</code>	categorie	Primaire, College, Lycee, Superieur
<code>filiere</code>	categorie	Mathematiques, Francais, Anglais, Sciences, Histoire, Informatique
<code>note_moyenne</code>	float	Note moyenne sur 20
<code>taux_assiduite</code>	float	Taux de presence en cours (0-100 %)
<code>heures_etude</code>	float	Heures d'etude par semaine
<code>acces_ressources</code>	int	0 = aucun, 1 = limite, 2 = correct, 3 = tres bon
<code>resultat_examen</code>	categorie	Reussi, Echoue, Non passe

Les variables categoriques sont encodees via **OrdinalEncoder** avec categories fixes, serialise dans `model.pkl`. Le modele predit : **En reussite**, **A surveiller** ou **A risque de décrochage**.

3.1.2. Validation des entrees

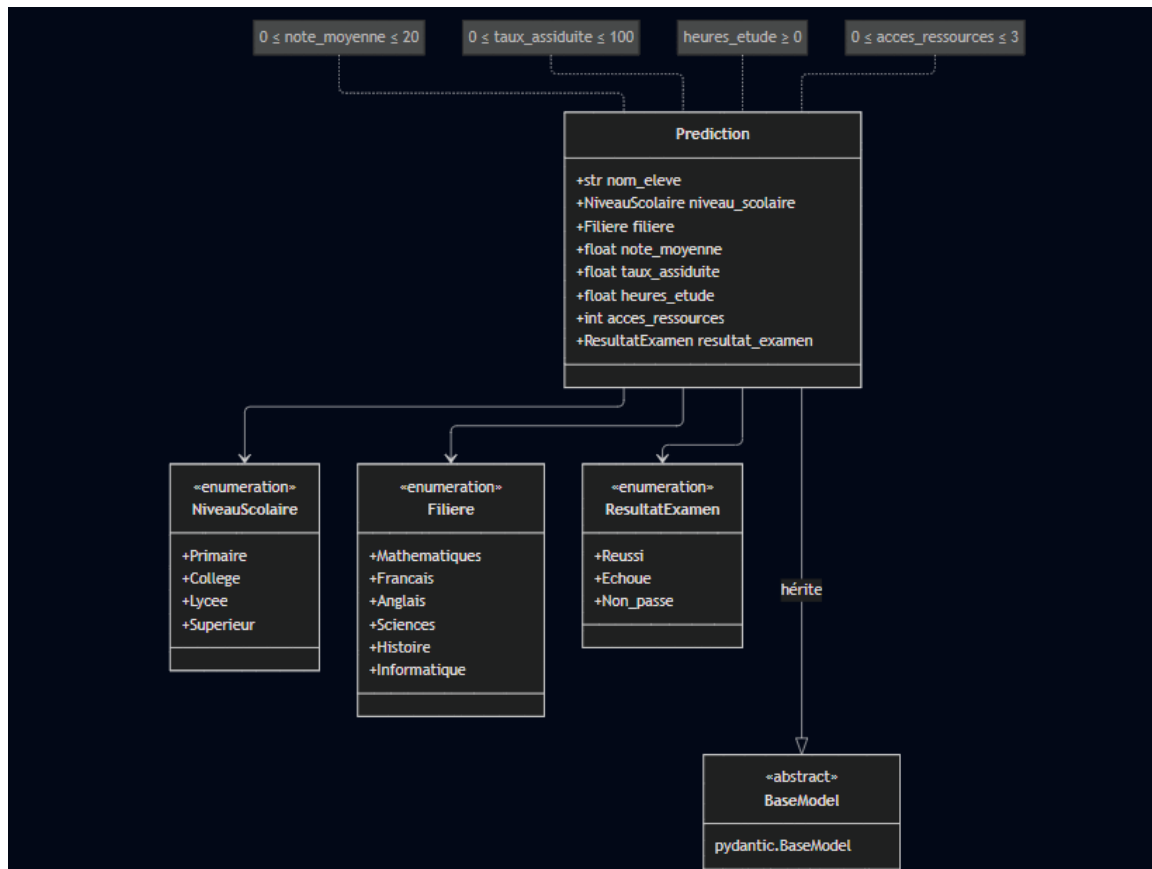
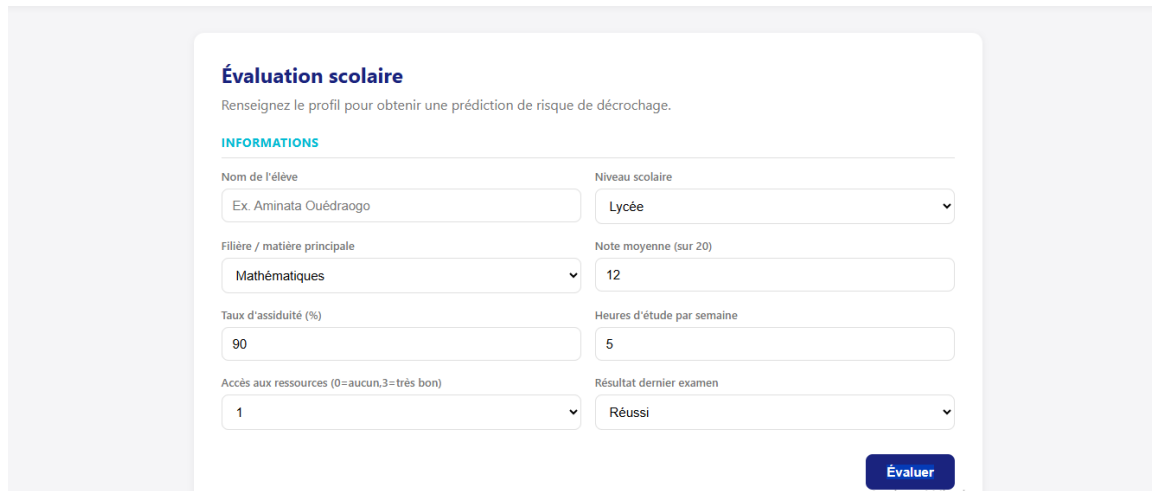


FIGURE 2 – Schema de validation Prediction(Pydantic v2)

3.2. Frontend — React

Le frontend est une application **React 18** servie par **Nginx** avec le formulaire de saisie complet :

- **Nom de l'eleve** : champ texte libre.
- **Niveau scolaire** : menu deroulant (Primaire / College / Lycee / Superieur).
- **Filiere / matiere principale** : menu deroulant (Mathematiques, Francais, Anglais, Sciences, Histoire, Informatique).
- **Note moyenne** : champ numerique sur 20.
- **Taux d'assidue** : curseur 0–100 %.
- **Heures d'etude par semaine** : champ numerique.
- **Acces aux ressources** : selecteur 0 (aucun) a 3 (tres bon).
- **Resultat dernier examen** : boutons radio (Reussi / Echoue / Non passe).



The screenshot shows a web form titled "Évaluation scolaire" (School Evaluation). Below the title is a subtitle: "Renseignez le profil pour obtenir une prédiction de risque de décrochage." (Fill in the profile to get a prediction of the risk of dropping out). A section header "INFORMATIONS" is followed by a grid of input fields. The fields are: "Nom de l'élève" (Student Name) with a text input containing "Ex. Aminata Ouédraogo"; "Niveau scolaire" (School Level) with a dropdown menu showing "Lycée"; "Filière / matière principale" (Main Subject) with a dropdown menu showing "Mathématiques"; "Note moyenne (sur 20)" (Average Grade) with a text input containing "12"; "Taux d'assiduité (%)" (Attendance Rate) with a text input containing "90"; "Heures d'étude par semaine" (Study Hours per Week) with a text input containing "5"; "Accès aux ressources (0=aucun,3=très bon)" (Access to resources) with a dropdown menu showing "1"; and "Résultat dernier examen" (Last exam result) with a dropdown menu showing "Réussi". A blue button labeled "Évaluer" (Evaluate) is located at the bottom right of the form.

FIGURE 3 – Interface frontend EduScore — formulaire de prediction

3.3. Base de donnees — PostgreSQL

La base de donnees **PostgreSQL 15** (instance RDS `db.t3.micro`) stocke :

- La table `predictions` : historique complet (donnees d'entree, resultat, score de confiance, timestamp).
- La table `users` (prevue pour l'authentification future).
- La table `items` : éléments de référence de l'application (libellé, valeur et date de création).

L'accès est gere via **SQLAlchemy** (ORM, pas de SQL brut). La `DATABASE_URL` est injectee depuis AWS SSM Parameter Store.



FIGURE 4 – Base de données PostgreSQL — tables

4. Conteneurisation et déploiement

4.1. Docker et Docker Compose

Chaque service possède son propre **Dockerfile** optimisé (build multi-stage). Le fichier `docker-compose.yml` à la racine permet de lancer la stack localement :

Listing 1 – Lancement local

```
1 docker compose up --build
```

```

PS C:\Users\hp\Desktop\S8\Sécurité & Gestion couts> cd Projet_securitecloud_used
PS C:\Users\hp\Desktop\S8\Sécurité & Gestion couts\Projet_securitecloud_used> docker compose up -d
[+] Running 3/3
  ✓ Container projet_securitecloud_used-db-1      Healthy      0.9s
  ✓ Container projet_securitecloud_used-backend-1 Running      0.0s
  ✓ Container projet_securitecloud_used-frontend-1 Running      0.0s
PS C:\Users\hp\Desktop\S8\Sécurité & Gestion couts\Projet_securitecloud_used>

```

FIGURE 5 – docker compose up — stack locale fonctionnelle

4.2. Pipeline CI/CD — GitHub Actions

Le workflow `.github/workflows/ci-cd.yml` s'exécute à chaque push sur main :

1. **Scan secrets** avec **Trufflehog** — bloque si secret détecté.
2. **Audit dependances** avec **pip-audit** — bloque si CVE critique.
3. **Tests unitaires** avec **pytest**.
4. **Build images Docker** (backend + frontend).
5. **Push vers ECR**.
6. **Deploiement ECS Fargate**.

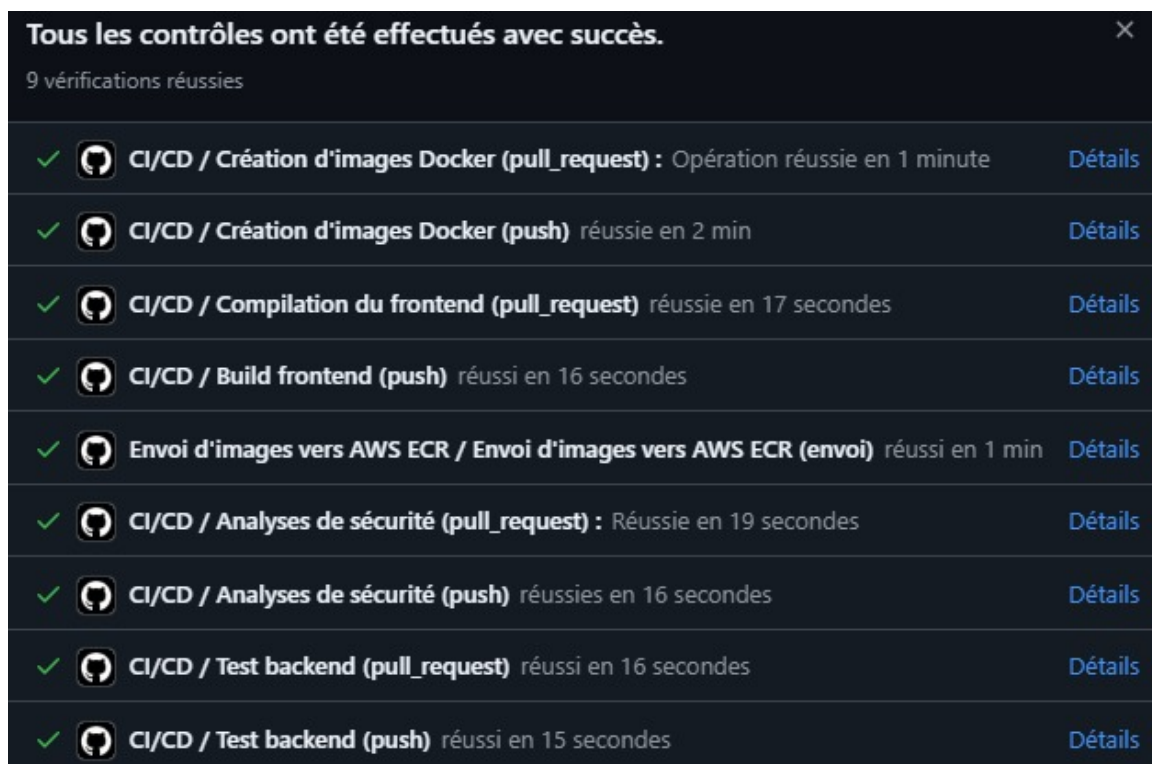


FIGURE 6 – Pipeline GitHub Actions — run réussi, tous les jobs au vert

Le scan réalisé avec TruffleHog a détecté uniquement des résultats unverified, qui n'ont pas été confirmés comme des secrets valides après analyse.

⇒ Les secrets identifiés ont été corrigés et déplacés vers des variables d'environnement.


```
(.venv) PS C:\Users\hp\Desktop\S8\Securité & Gestion cours\Projet_securitecloud_used> docker run --rm -v ${PWD}:/repo trufflsec
curity/trufflehog filesystem /repo
Line: 22

Found unverified result 🚨🔑?
Verification issue: i/o timeout
(🔑 Using cached verification)
Detector Type: Postgres
Decoder Type: PLAIN
Raw result: postgres://postgres:KoneBakouan01@database-1.cby4yi2mm3xm.eu-west-1.rds.amazonaws.com:5432
Sslmode: require
Host: database-1.cby4yi2mm3xm.eu-west-1.rds.amazonaws.com:5432
Username: postgres
Database: postgres
File: /repo/backend-td.json
Line: 23

Found unverified result 🚨🔑?
Verification issue: i/o timeout
(🔑 Using cached verification)
```

FIGURE 7 – Scan Trufflehog en CI — aucun secret expose detecte

4.3. Deploiement sur AWS

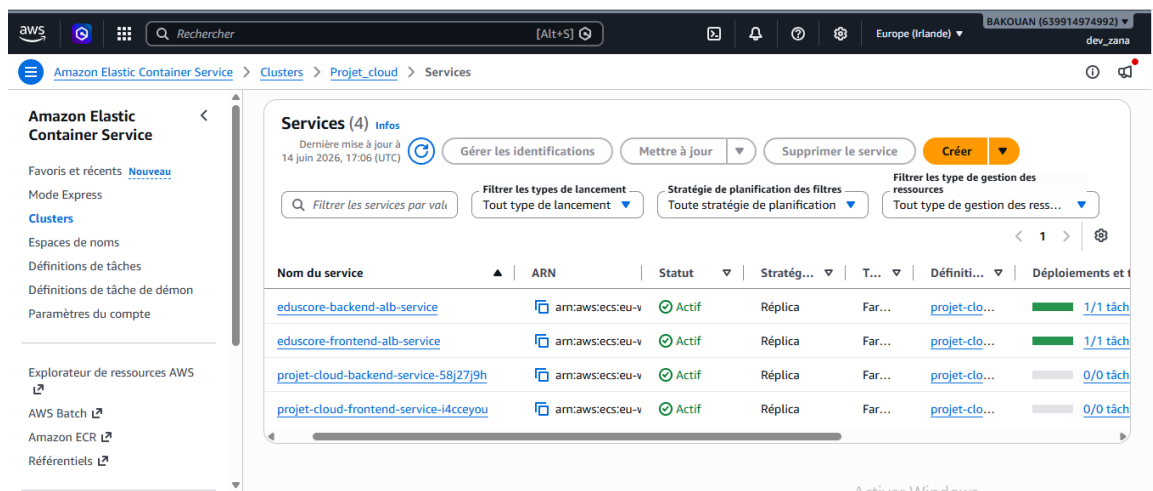
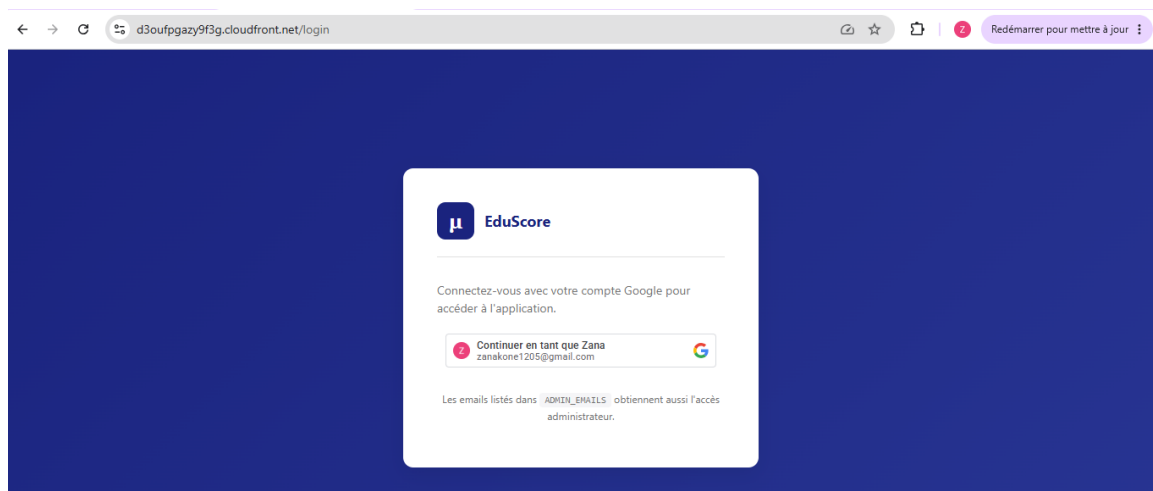


FIGURE 8 – Console AWS ECS — services Backend et Frontend RUNNING

FIGURE 9 – Application deployée — <https://d3oufpgazy9f3g.cloudfront.net> accessible en HTTPS

5. Securite

5.1. Execution du plan de securite (Livrable 1)

TABLE 3 – Tableau de bord securite — plan L1 vs realisation L2

Mesure prevue (L1)	Faite ?	Preuve / Ecart
Secrets en variables d'environnement (GitHub Secrets)	Oui	Fig. 10
.gitignore + .env.example sans valeurs	Oui	Present dans le repo
Scan Trufflehog en CI	Oui	Fig. 7
Audit dependances pip-audit	Oui	Step CI actif
Scan image Trivy en CI	Oui	Step CI actif
HTTPS en production (CloudFront + ACM)	Oui	Fig. 9
Validation Pydantic v2 sur /predict	Oui	Schema Prediction
ORM SQLAlchemy (pas de SQL brut)	Oui	Pas d'injection SQL
S3 Block Public Access active	Oui	Console AWS
RDS sous-reseau prive + chiffrement	Oui	VPC configuration
SSM Parameter Store pour les secrets	Oui	Taches ECS
IAM moindre privilege par role	Oui	Roles ECS configures
Rate-limit 20 req/min/IP sur /predict	Oui	SlowAPI configure
Google OAuth + JWT	Oui	Google OAuth et JWT .
WAF AWS	Non	Hors budget
Multi-AZ RDS (failover actif)	Partiel	Backup active, failover non configure

5.2. Gestion des secrets

- **Secrets CI/CD** (GitHub Secrets) : AWS_ROLE_ARN, ECR_REGISTRY, AWS_REGION.
- **Secrets applicatifs** (SSM Parameter Store) : JWT_SECRET, DATABASE_URL — injectes dans les conteneurs ECS via role IAM.



FIGURE 10 – GitHub Secrets — secrets CI/CD proteges (aucune valeur visible)

6. Coûts

6.1. Coûts reels vs estimés

TABLE 4 – Comparaison coûts estimés (L1) vs coûts reels (L2)

Poste	Service	Estime L1	Reel L2
Compute Backend	ECS Fargate (2 taches)	15,00 \$	≈2-3 \$
Compute Frontend	ECS Fargate (2 taches)	8,00 \$	≈1-2 \$
Load Balancer	ALB	18,00 \$	≈2-3 \$
CDN	CloudFront	0,00 \$	≈1,50 \$
Base de données	PostgreSQL RDS t3.micro	12,00 \$	≈2-3 \$
Stockage modele	S3 (~500 MB)	0,50 \$	≈0,50 \$
Secrets	SSM Parameter Store	0,00 \$	0,00 \$
Monitoring	CloudWatch	5,00 \$	≈<1 \$
Transfert	Data Transfer	2,00 \$	≈1,50 \$
Divers	ECR, Security Groups	1,00 \$	≈1,00 \$
Total		61,50 \$	≈9,52 \$.

Le coût reel est inférieur de **84,5 %** à l'estimation grâce à l'absence de NAT Gateway et l'optimisation des tâches ECS.

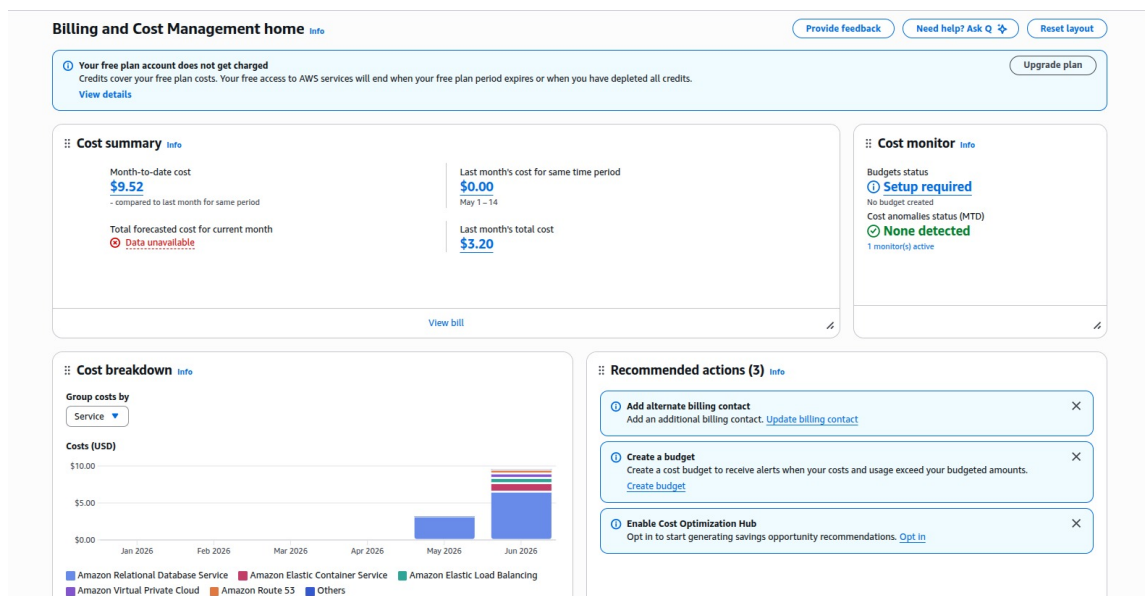


FIGURE 11 – AWS Cost Explorer — coûts reels du projet par service

7. Difficultes rencontrees et solutions

TABLE 5 – Difficultes et solutions

Difficulte	Cause	Solution apportee
Health check ALB en echec	Backend met 30s a charger le modele S3	<code>healthCheckGracePeriodSeconds=60</code> dans la tache ECS
Connexion RDS refusee	Security Group RDS trop restrictif	Regle ingress depuis le SG backend
Trufflehog faux positifs	Cles d'exemple dans <code>.env.example</code>	Patterns d'exclusion dans <code>.trufflehog.yml</code>
Timeout <code>/predict</code>	Modele recharge a chaque requete	Cache memoire au demarrage (<code>startup event</code>)
Encodage categoriel	Niveaux/filieres non reconnus en prod	<code>OrdinalEncoder</code> fixe serialise dans <code>model.pkl</code>
CORS bloque React	Domaine CloudFront absent de la whitelist	<code>CORSMiddleware</code> avec domaine CloudFront

8. Conclusion

Le Livrable 2 concretise le plan defini dans le Livrable 1 : **EduScore** est deploye, accessible en HTTPS via <https://d3oufpgazy9f3g.cloudfront.net>, et produit des predictions de decrochage scolaire en temps reel a partir du profil complet de l'eleve.

- **Architecture** : variante budget respectee, avec CloudFront ajoute pour les performances et la securite.
- **Modele ML** : 8 features couvrant le profil scolaire complet, 3 categories de sortie avec score de confiance.
- **Securite** : 14 des 16 mesures implementees .
- **CI/CD** : Trufflehog, pip-audit actif sur chaque push.
- **Couts** : 9,52 \$/mois, soit inferieur a 84,5 % par rapport a l'estimation.

Perspectives. Prochaine iteration : WAF AWS, tests end-to-end en CI.

Annexes

A — Structure du projet

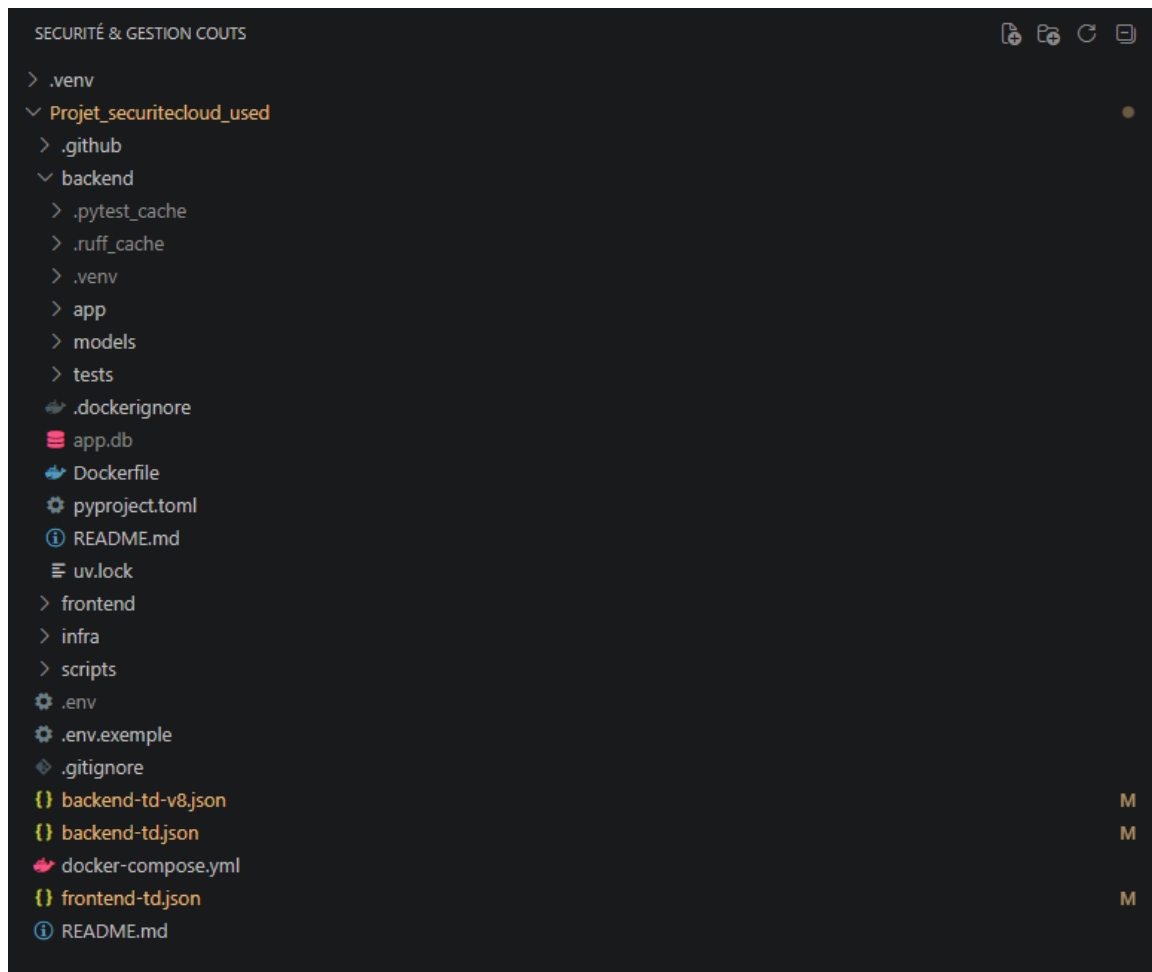


FIGURE 12 – Arborescence du projet